

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Course Name :
Software Engineering

Project Title :
Appointment Scheduling System

Supervisor:	Dr.Haya Samaana		
Student:	Hamdi Balati	Abd Alrahaman Abd Alfatah	Ammar Mohammed
Section:	1-2	12-2	12-2
Reg No. :	12240270	12217086	12027945
Work Percentage 100%	33.3%	33.3%	33.3%

1.Introduction:

The Appointment System is a desktop application developed using Java to help users and administrators manage appointments efficiently. The system provides several functionalities such as booking appointments, canceling appointments, modifying bookings, and viewing available appointments through a graphical user interface (GUI).

The application supports different appointment types and booking rules, including individual, group, urgent, virtual, assessment, and follow-up appointments. In addition, the system includes authentication features for users and administrators, input validation, email notification support, and appointment management services.

The project was developed following Object-Oriented Programming (OOP) principles and software engineering best practices. Several design patterns were applied in the system, such as the Strategy Pattern and Observer Pattern, to improve flexibility and maintainability.

In Phase 2, the project was enhanced using static code analysis tools such as SonarCloud and code coverage tools such as JaCoCo. Multiple code smells and maintainability issues were identified and refactored to improve code quality, readability, and system reliability.

The Appointment System is designed to simplify the management of appointments for both users and administrators. The system provides a user-friendly graphical interface that allows users to interact with the application

2. Main Features

- User and administrator login system.
- Add and manage appointments.
- Book, cancel, and modify appointments.
- Display available appointments.
- Apply different booking rules depending on appointment type.
- Send email notifications.
- Validate user inputs before processing operations.

Technologies Used

- Java
- Java Swing (GUI)
- Maven
- JUnit 5
- Mockito
- SonarCloud
- JaCoCo

Design Patterns Used

- Strategy Pattern
- Observer Pattern

3. SonarCloud Analysis Before Refactoring:

SonarQube cloud Favorite Projects Assigned Issues Explore Upgrade

Filters

Quality Gate

- Passed 0
- Failed 0

Reliability

- A 1
- B 0
- C 0
- D 0
- E 0

Security

- A 1
- B 0
- C 0
- D 0
- E 0

Security Review

- A ≥ 80% 0

Favorite Projects

Projects you marked as favorites

Search projects Perspective Overall Status Sort by Name 1 projects

HamdiBalati-org / Software-Engineering- New Public

Last analysis: 4/23/2026, 6:02 PM - 1.7k Lines of Code - Java, XML

Not computed

Security 0 Reliability 0 Maintainability 56 Hotspots Reviewed 0.0% Duplications 0.0%

1 of 1 shown

© 2018-2026 SonarSource Sàrl. All rights reserved. Terms Pricing Privacy Cookie Policy Security Community Documentation Contact us Status About

SonarQube cloud Favorite Projects Assigned Issues Explore Upgrade

AppointmentSystem Project

HamdiBalati-org / AppointmentSystem / Summary

Summary main

Public - 1.6k Lines of Code - Version 0.0.1-SNAPSHOT - Last analysis 52 seconds ago - e5e78ae3 View on GitHub

Quality Gate: Sonar way Passed

New Code Overall Code

New code Since 16 days ago

New Issues 4

No conditions set

Accepted Issues 0

Valid issues that were not fixed

Coverage 74.27%

No conditions set on 898 New Lines to cover

Duplications 0.0%

Required: ≤ 3.0%

Security Hotspots 1

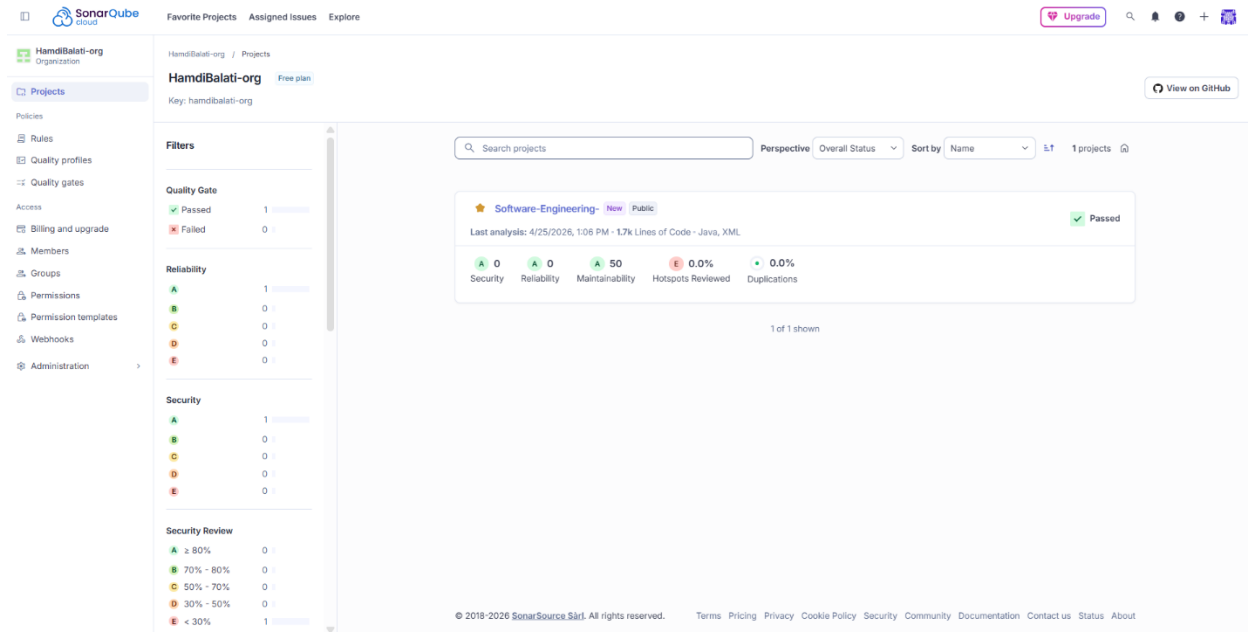
No conditions set

Summary Issues Architecture Security hotspots Reporting Measures Activity Policies Intended architecture Quality profiles Quality gate Project Pull Requests Branches Code Project Information

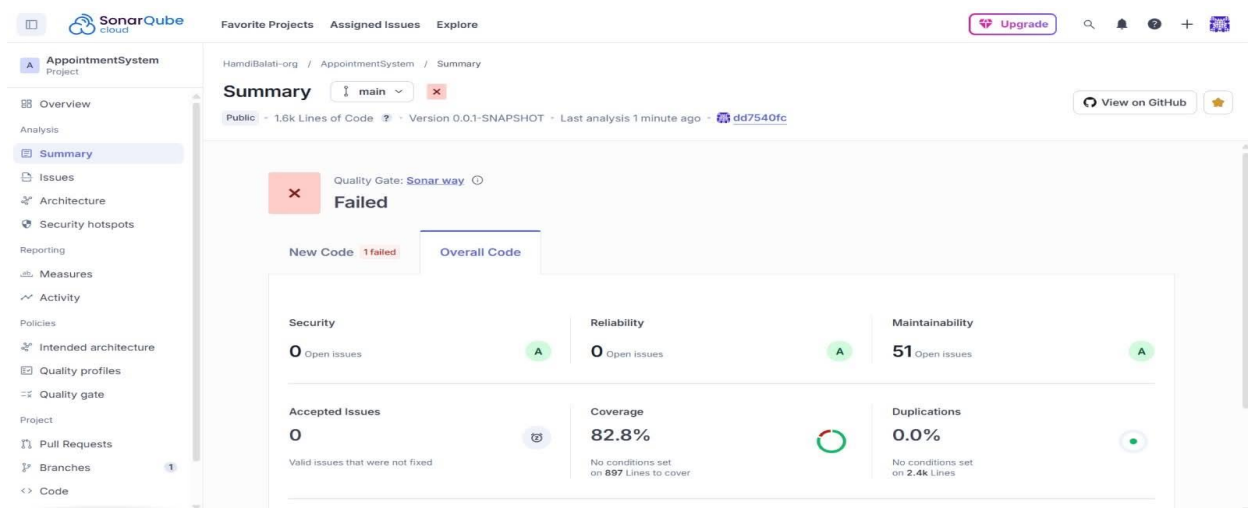
The initial SonarCloud analysis revealed several maintainability issues, code smells, and security hotspot warnings in the project. During the refactoring process, multiple improvements were applied to enhance code quality and system design. Additional unit tests were also

implemented to improve test coverage. As a result, the project Quality Gate status successfully passed, the number of issues was reduced, and the coverage increased from 74.27% to 82.8%, exceeding the required minimum threshold of 80%.

4. SonarCloud Analysis After Refactoring:



The screenshot shows the SonarCloud dashboard for the organization 'HamdiBalati-org'. The left sidebar contains navigation links for Projects, Policies, Rules, Quality profiles, Quality gates, Access, Billing and upgrade, Members, Groups, Permissions, Permission templates, Webhooks, and Administration. The main content area displays the 'HamdiBalati-org' project with a 'Free plan' badge and a 'View on GitHub' link. The 'Filters' section on the left shows 'Quality Gate' with 'Passed' (1) and 'Failed' (0) counts, and 'Reliability' with 'A' (1), 'B' (0), 'C' (0), 'D' (0), and 'E' (0) counts. The 'Security' section shows 'A' (1), 'B' (0), 'C' (0), 'D' (0), and 'E' (0) counts. The 'Security Review' section shows 'A' (1), 'B' (0), 'C' (0), 'D' (0), and 'E' (0) counts. The main content area shows a search bar, 'Perspective' and 'Overall Status' dropdowns, and 'Sort by' dropdown. The project 'Software-Engineering- New Public' is listed with a 'Passed' status. The last analysis was on 4/25/2026 at 1:08 PM, with 1.7k Lines of Code in Java, XML. The metrics are: Security (A 0), Reliability (A 0), Maintainability (A 50), Hotspots Reviewed (E 0.0%), and Duplications (0.0%).



The screenshot shows the SonarCloud dashboard for the project 'AppointmentSystem'. The left sidebar contains navigation links for Overview, Analysis, Summary, Issues, Architecture, Security hotspots, Reporting, Measures, Activity, Policies, Intended architecture, Quality profiles, Quality gate, Project, Pull Requests, Branches, and Code. The main content area displays the 'AppointmentSystem' project with a 'Summary' tab selected. The 'Summary' section shows a 'Failed' status with a 'Quality Gate: Sonar.way' badge. The 'New Code' section shows '1 failed' and 'Overall Code' section shows 'Overall Code'. The metrics are: Security (0 Open issues, A), Reliability (0 Open issues, A), Maintainability (51 Open issues, A), Accepted Issues (0, Valid issues that were not fixed), Coverage (82.8%, No conditions set on 897 Lines to cover), and Duplications (0.0%, No conditions set on 2.4k Lines).

After applying the refactoring process, the project quality improved significantly. Several code smells and maintainability issues were resolved, and the system design became cleaner and more maintainable. Additional unit tests were added to increase the test coverage from 74.27% to 82.8%, successfully exceeding the required minimum threshold of 80%. The final SonarCloud analysis also showed improved project stability with reduced issues and better overall code quality.

****Important notes**:**

At the beginning of working on Phase 2, we fixed 5 errors, but we didn't know that the coverage needed to be inside the sonar cloud. So, we then increased the coverage on the sonar cloud, and the percentage appeared as 74.27%. We then increased it to 82.8% by adding test cases, which is what we wanted—a percentage above 80%.

4. Code Smells and Refactoring :

Resolved elements:

Type	issue	class	Solution
Bad Smell	Duplicated string literals ("Pending", "Confirmed", "FULL")	Appointment	Created constant variables and reused them
Bad Smell	Duplicated UI text ("Add Appointment")	AppointmentsGUI	Defined BTN_ADD_APPOINTMENT constant
Bad Smell	Unused parameter type	Appointment	Used the parameter inside the constructor
Vulnerability	Non-serializable field inside serializable class	AppointmentsGUI	Marked the field as transient
Bug	Static reference / static fields issue	GUI classes	Converted methods and fields to proper instance usage

Below this table is an explanation of the problems.

4.1 Issue 1: Duplicated String Literals ("Pending" / "Confirmed" / "FULL")

Problem:

The same string values were repeated multiple times throughout the project code.

Reason:

This violated the DRY (Don't Repeat Yourself) principle.

Impact:

- Harder code maintenance
- Increased risk of inconsistencies
- Reduced readability

Refactoring Solution:

Constant variables were created and reused instead of duplicating hardcoded string literals.

Result:

The code became cleaner, more maintainable, and easier to update.

4.2 Issue 2: Duplicated UI Text ("Add Appointment")

Problem:

The text "Add Appointment" appeared multiple times in the GUI classes.

Reason:

Constants were not used for reusable UI labels.

Impact:

- Inconsistent user interface text

- Difficult maintenance and updates

Refactoring Solution

A constant named `BTN_ADD_APPOINTMENT` was defined and reused across the GUI.

Result:

The GUI code became more organized and maintainable.

Issue 3: Non-Serializable Field in Serializable Class

Problem:

A non-static field existed inside a serializable class.

Reason:

Serializable classes should not contain non-serializable fields.

Impact:

- Serialization problems
- Possible runtime exceptions

Refactoring Solution

The field was marked as transient.

Result:

Serialization issues were resolved successfully.

Issue 4: Static Fields Modified Inside Instance Methods

Problem:

Static variables were modified inside non-static methods.

Reason:

Poor design caused mixing between static and instance members.

Impact:

- Unexpected behavior
- Difficult debugging and maintenance

Refactoring Solution:

The unnecessary static keyword was removed from the fields.

Result:

The class design became cleaner and more stable.

Issue 5: Unused Parameter (type)

Problem:

A constructor parameter named type was declared but never used.

Reason:

Incomplete implementation.

Impact:

- Confusing code structure
- Reduced readability

Refactoring Solution:

The parameter was integrated properly into the constructor logic.

Result:

The warning disappeared and the implementation became clearer.

Issue 6: Static Reference Error

Problem:

Static methods were accessing non-static fields after refactoring.

Reason:

Methods were not updated after removing static fields.

Impact:

- Compilation errors
- System instability

Refactoring Solution:

The affected methods were converted from static methods to instance methods.

Result : The project compiled successfully and the design consistency improved.

****Final Refactoring Outcome:**

After completing the refactoring process:

- Code maintainability improved significantly.
- Code duplication was reduced.
- Design and serialization issues were fixed.
- The overall project became cleaner, safer, and easier to maintain.

5. Design Pattern Used:

The project implements the Strategy Design Pattern to manage different appointment validation rules.

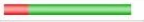
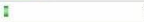
The BookingRuleStrategy interface defines the common validation behavior, while multiple concrete strategy classes such as UrgentRule, GroupRule, AssessmentRule, and FollowUpRule provide different validation implementations depending on the appointment type.

This design pattern improves flexibility, scalability, and maintainability by allowing new booking rules to be added without modifying existing system logic.

- BookingRuleStrategy , UrgentRule, GroupRule, AssessmentRule FollowUpRule and InPersonRule

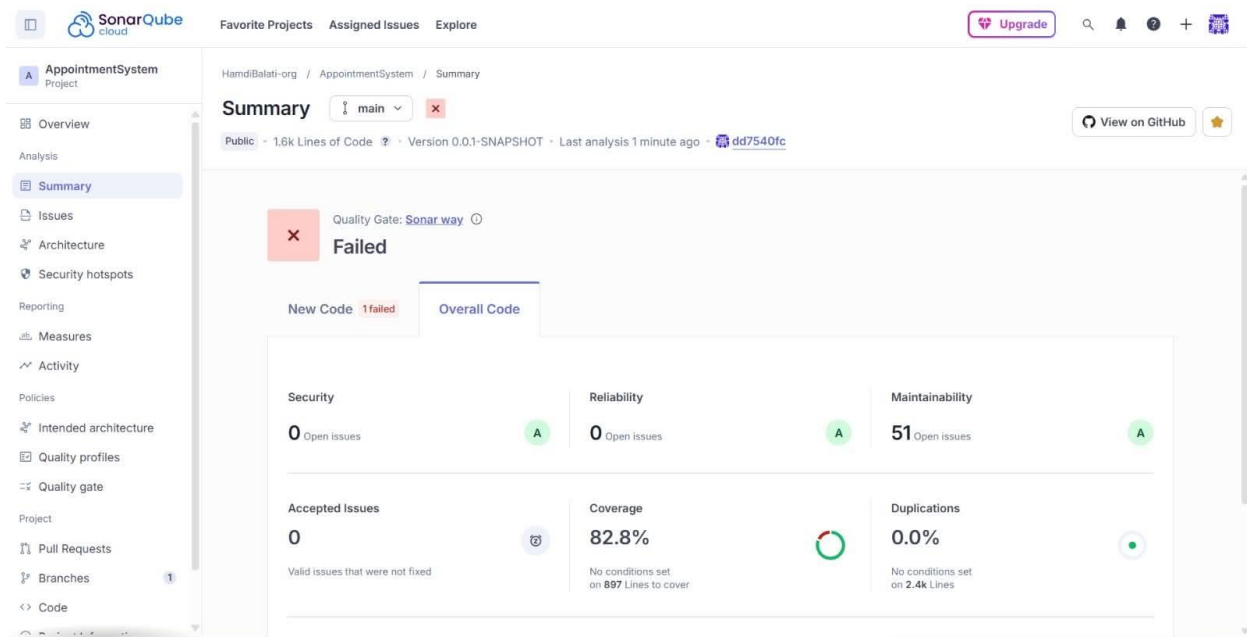
6. Coverage Result Report:

6.1 The following figure shows the final JaCoCo coverage report for the Appointment System project.

AppointmentSystem												
AppointmentSystem												
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes				
com.hamdi.appointments.domain		98%		93%	1 31	1 75	0 23	0 4				
com.hamdi.appointments.gui		80%		58%	52 119	124 507	16 49	0 5				
com.hamdi.appointments.notification		100%		100%	0 14	0 46	0 12	0 4				
com.hamdi.appointments.repository		100%		100%	0 9	0 16	0 5	0 1				
com.hamdi.appointments.service		96%		94%	5 68	8 217	0 23	0 2				
com.hamdi.appointments.strategy		100%		100%	0 27	0 21	0 21	0 7				
com.hamdi.appointments.UandAandA		96%		50%	2 3	1 16	0 1	0 1				
Total	497 of 3,791	86%	66 of 268	75%	60 271	134 898	16 134	0 24				

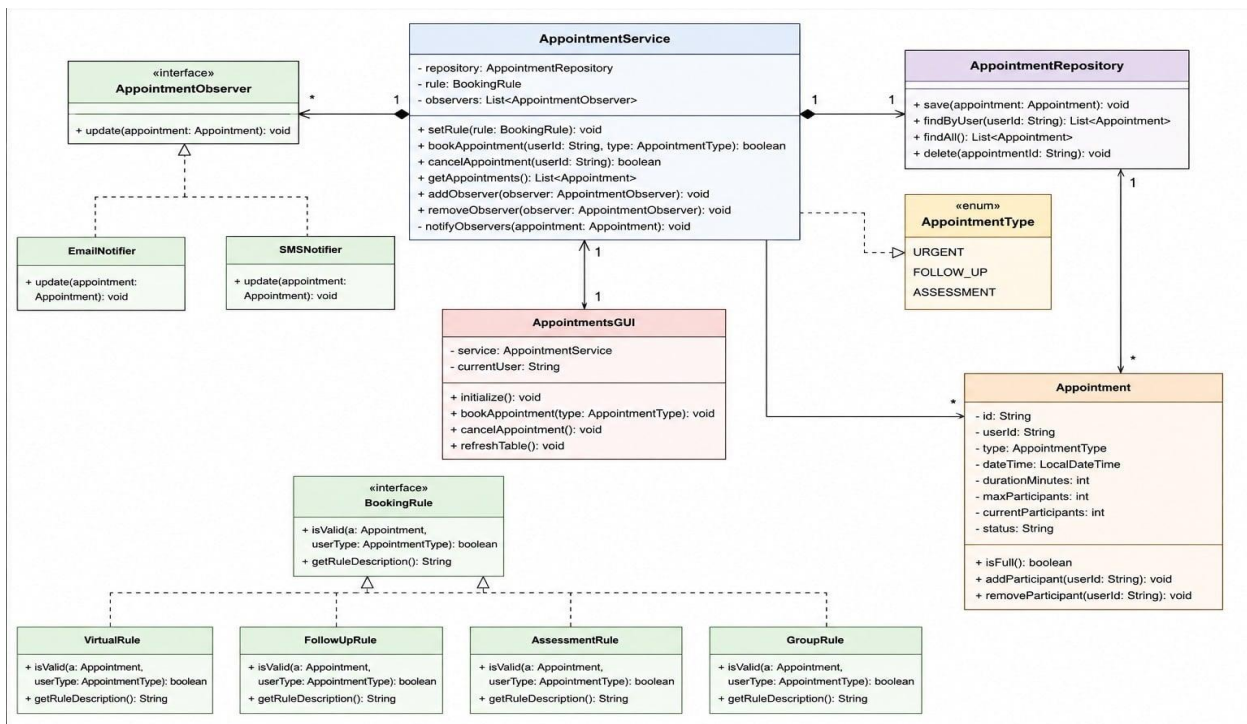
The JaCoCo coverage report demonstrates that the project achieved a final instruction coverage of 86%, which is above the required minimum threshold (>80%). The report also shows high test coverage across most project packages, including the service, repository, domain, notification, and strategy layers, indicating improved software reliability and testing quality.

6.2 Final SonarCloud Coverage Result:



The final SonarCloud analysis confirmed that the project coverage reached 82.8%, which is above the required minimum threshold (>80%).

7.UML Diagram:



The UML class diagram illustrates the overall structure of the Appointment System project. It shows the relationships between the main classes, including inheritance, associations, and interactions between the domain, service, repository, GUI, and notification layers. The diagram helps visualize the system architecture and improves understanding of the project design.

8. CI Result from GitHub Actions:

4 workflow runs				Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Update maven.yml	main	1 minute ago	21s	...			
Java CI with Maven #4: Commit ccdf10e pushed by hamdibalati2-Eng								
✗	Update maven.yml	main	2 minutes ago	34s	...			
Java CI with Maven #3: Commit a70c9ee pushed by hamdibalati2-Eng								
✗	Update maven.yml	main	9 minutes ago	33s	...			
Java CI with Maven #2: Commit be3631a pushed by hamdibalati2-Eng								
✗	Create maven.yml	main	13 minutes ago	21s	...			
Java CI with Maven #1: Commit 6c46f68 pushed by hamdibalati2-Eng								

GitHub Actions was configured to automate the Maven build process for the project. After fixing the workflow configuration, the CI pipeline ran successfully and the project was built without errors. This confirms that the project can be automatically integrated and verified through GitHub Actions.

9. Conclusion :

In conclusion, the Appointment System project was successfully improved through static analysis, refactoring, and additional unit testing. SonarCloud helped identify code smells and maintainability issues, while JaCoCo was used to evaluate test coverage. After applying the required improvements, the project achieved better code quality, cleaner design, and a final coverage percentage above the required threshold (>80%).